
1.7 An Example Program

In this section, we present another example of a Java class, which illustrates many of the constructs defined thus far in this chapter. This `CreditCard` class defines credit card objects that model a simplified version of traditional credit cards. They store information about the customer, issuing bank, account identifier, credit limit, and current balance. They do not charge interest or late payments, but they do restrict charges that would cause a card's balance to go over its credit limit. We also provide a static main method as part of this class to demonstrate its use.

The primary definition of the `CreditCard` class is given in Code Fragment 1.5. We defer until Code Fragment 1.6 the presentation of the main method, and in Code Fragment 1.7 we show the output produced by the main method. Highlights of this class, and underlying techniques that are demonstrated, include:

- The class defines five instance variables (lines 3–7), four of which are declared as **private** and one that is **protected**. (We will take advantage of the **protected** balance member when introducing inheritance in the next chapter.)
- The class defines two different constructor forms. The first version (beginning at line 9) requires five parameters, including an explicit initial balance for the account. The second constructor (beginning at line 16) accepts only four parameters; it relies on use of the special **this** keyword to invoke the five-parameter version, with an explicit initial balance of zero (a reasonable default for most new accounts).
- The class defines five basic accessor methods (lines 20–24), and two update methods (charge and makePayment). The charge method relies on conditional logic to ensure that a charge is rejected if it would have resulted in the balance exceeding the credit limit on the card.
- We provide a **static** utility method, named `printSummary`, in lines 37–43.
- The main method includes an array, named `wallet`, storing `CreditCard` instances. The main method also demonstrates a **while** loop, a traditional **for** loop, and a for-each loop over the contents of the wallet.
- The main method demonstrates the syntax for calling traditional (nonstatic) methods—charge, getBalance, and makePayment—as well as the syntax for invoking the static `printSummary` method.

```

1  public class CreditCard {
2      // Instance variables:
3      private String customer;    // name of the customer (e.g., "John Bowman")
4      private String bank;        // name of the bank (e.g., "California Savings")
5      private String account;     // account identifier (e.g., "5391 0375 9387 5309")
6      private int limit;          // credit limit (measured in dollars)
7      protected double balance;   // current balance (measured in dollars)
8      // Constructors:
9      public CreditCard(String cust, String bk, String acnt, int lim, double initialBal) {
10         customer = cust;
11         bank = bk;
12         account = acnt;
13         limit = lim;
14         balance = initialBal;
15     }
16     public CreditCard(String cust, String bk, String acnt, int lim) {
17         this(cust, bk, acnt, lim, 0.0);    // use a balance of zero as default
18     }
19     // Accessor methods:
20     public String getCustomer() { return customer; }
21     public String getBank() { return bank; }
22     public String getAccount() { return account; }
23     public int getLimit() { return limit; }
24     public double getBalance() { return balance; }
25     // Update methods:
26     public boolean charge(double price) {    // make a charge
27         if (price + balance > limit)        // if charge would surpass limit
28             return false;                  // refuse the charge
29         // at this point, the charge is successful
30         balance += price;                   // update the balance
31         return true;                       // announce the good news
32     }
33     public void makePayment(double amount) { // make a payment
34         balance -= amount;
35     }
36     // Utility method to print a card's information
37     public static void printSummary(CreditCard card) {
38         System.out.println("Customer = " + card.customer);
39         System.out.println("Bank = " + card.bank);
40         System.out.println("Account = " + card.account);
41         System.out.println("Balance = " + card.balance); // implicit cast
42         System.out.println("Limit = " + card.limit);     // implicit cast
43     }
44     // main method shown on next page...
45 }

```

Code Fragment 1.5: The CreditCard class.

```

1  public static void main(String[ ] args) {
2      CreditCard[ ] wallet = new CreditCard[3];
3      wallet[0] = new CreditCard("John Bowman", "California Savings",
4                                  "5391 0375 9387 5309", 5000);
5      wallet[1] = new CreditCard("John Bowman", "California Federal",
6                                  "3485 0399 3395 1954", 3500);
7      wallet[2] = new CreditCard("John Bowman", "California Finance",
8                                  "5391 0375 9387 5309", 2500, 300);
9
10     for (int val = 1; val <= 16; val++) {
11         wallet[0].charge(3*val);
12         wallet[1].charge(2*val);
13         wallet[2].charge(val);
14     }
15
16     for (CreditCard card : wallet) {
17         CreditCard.printSummary(card);           // calling static method
18         while (card.getBalance() > 200.0) {
19             card.makePayment(200);
20             System.out.println("New balance = " + card.getBalance());
21         }
22     }
23 }

```

Code Fragment 1.6: The main method of the CreditCard class.

```

Customer = John Bowman
Bank = California Savings
Account = 5391 0375 9387 5309
Balance = 408.0
Limit = 5000
New balance = 208.0
New balance = 8.0
Customer = John Bowman
Bank = California Federal
Account = 3485 0399 3395 1954
Balance = 272.0
Limit = 3500
New balance = 72.0
Customer = John Bowman
Bank = California Finance
Account = 5391 0375 9387 5309
Balance = 436.0
Limit = 2500
New balance = 236.0
New balance = 36.0

```

Code Fragment 1.7: Output from the Test class.